

Diplomado virtual PROGRAMACIÓN EN JAVASCRIPT

Guía didáctica – Módulo 4



```
382     (this.paused = true) {
383     if (this.$element.find('.next, .prev').length &&
384         this.cycle(true)
385     }
386     this.interval = clearInterval(this.interval)
387     return this
388 }
389
390 Carousel.prototype.next = function () {
391     if (this.sliding) return
392     return this.slide('next')
393 }
394
395 Carousel.prototype.prev = function () {
396     if (this.sliding) return
397     return this.slide('prev')
398 }
399
400 Carousel.prototype.slide = function (type, next) {
401     var $active = this.$element.find('.item.active')
402     var $next = next || this.getItemForDirection(
403         type, 'next' ? 'left' : 'right'
404     )
405     var isCycling = this.interval
406     var direction = type == 'next' ? 'left' : 'right'
407     var fallback = type == 'next' ? 'first' : 'last'
408     var that = this
409
410     if (!$next.length) {
411         if (!this.options.wrap) return
412         $next = this.$element.find('.item')[fallback]()
413     }
414     if ($next.hasClass('active')) return (this.sliding)
415
416     var relatedTarget = $next[0]
417     var slideEvent = $.Event('slide.bs.carousel', {
418         relatedTarget: relatedTarget,
419         direction: direction
420     })
421     this.$element.trigger(slideEvent)
422     this.$element.trigger(slideEvent)
423 }
```

Imagen: Pixabay

Guía didáctica

HTML, Bootstrap y DOM

JS

Imagen: Pixabay



Competencia específica

Se espera que, con los temas abordados en la guía didáctica del módulo 4: HTML, Bootstrap y DOM, el estudiante logre la siguiente competencia específica:

Aplicar el concepto de DOM para la manipulación dinámica del HTML de los sitios web por medio de JavaScript.



Contenidos temáticos



Imagen: Freepik

Los siguientes son los contenidos temáticos para desarrollar en la guía didáctica del módulo 4: HTML, Bootstrap y DOM:

1

HTML

2

Bootstrap

3

DOM (manipulación de HTML)

4

Eventos



Tema 1:

HTML

HTML

HTML, siglas de *hypertext markup language* (lenguaje de marcado de hipertexto), es el lenguaje estándar utilizado para **estructurar** y **mostrar contenido** en la **web**.

Sus primeros conceptos fueron desarrollados a finales de la década de 1980 y su primera versión oficial fue lanzada en 1991.

Es el lenguaje fundamental de la *world wide web* y su estructura permite que las páginas web sean visualizadas en cualquier navegador compatible en todo tipo de dispositivos.

Presenta las siguientes características, que veremos con detalle a continuación.

Lenguaje de
marcado

Hipertexto

Independiente

Extensible



Características

Lenguaje de marcado

HTML utiliza un conjunto de etiquetas o «marcas» (*tags*) para estructurar el contenido. Estas etiquetas son interpretadas por los navegadores, que las transforman en el contenido visible para el usuario.

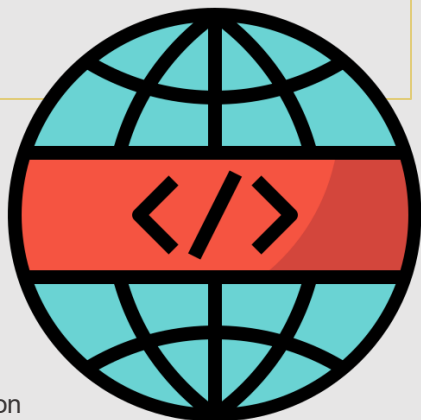


Imagen: Flaticon

Hipertexto

HTML permite la creación de enlaces o hipervínculos, que conectan documentos entre sí y forman una red interconectada de información en la web. Los enlaces son clave en HTML, ya que permiten la navegación fluida entre páginas web, formando la esencia de la web como la conocemos.

Independiente

Las páginas HTML se muestran de manera compatible en diferentes sistemas operativos, navegadores y dispositivos, como computadoras, tabletas y *smartphones*. La mayoría de las veces, el código HTML es el mismo sin importar el entorno donde se ejecute.

Extensible

HTML está diseñado para ser compatible con otros lenguajes y tecnologías web. Con frecuencia, se combina con CSS (para estilos) y JavaScript (para interactividad), creando páginas web dinámicas y atractivas.

Estructura

Los documentos HTML se debe crear con la extensión **.HTML**.

Las etiquetas **abren** y **cierran** normalmente: **<html></html>**, **<p></p>**.

Las etiquetas pueden contener **atributos** para personalizarse.

HTML permite integrarse con **JavaScript** y **CSS**.

```
1  <!DOCTYPE html>
2  <html lang="es">
3
4  <head>
5      <meta charset="UTF-8" />
6      <title>Título de la Página</title>
7  </head>
8
9  <body>
10     <h1>Encabezado Principal</h1>
11     <p>Este es un párrafo de ejemplo.</p>
12 </body>
13
14 </html>
```


Etiquetas esenciales

Las etiquetas en **HTML** son la base fundamental de la estructuración de sitios web por medio de **HTML**, algunas de las etiquetas más importantes son:

- **Estructuración**
- **Texto**
- **Tablas**
- **Multimedia**
- **Formularios**

```
1 <!DOCTYPE html>
2 <html lang="es">
3
4 <head>
5     <meta charset="UTF-8" />
6     <title>Título de la Página</title>
7 </head>
8
9 <body>
10     <h1>Encabezado Principal</h1>
11     <p>Este es un párrafo de ejemplo.</p>
12 </body>
13
14 </html>
```

← → ↻ ⓘ 127.0.0.1:5500/index.html

Encabezado Principal

Este es un párrafo de ejemplo.

Etiquetas de estructuración

Las etiquetas básicas de estructuración son las etiquetas más simples que permiten definir la estructura base de un sitio web.

<html> es la etiqueta contenedora de todo el contenido HTML.

<head> define la cabecera del sitio web.

<body> representa el contenido principal y central de un sitio web.

<footer> define la parte final de un sitio web.

```
1  <!DOCTYPE html>
2  <html lang="es">
3
4  <head>
5      <meta charset="UTF-8" />
6      <title>Título de la Página</title>
7  </head>
8
9  <body>
10
11  <footer>
12
13  </footer>
14
15  </body>
16
17  </html>
```

Etiquetas de texto

HTML tiene diferentes etiquetas para la creación de componentes de texto según la necesidad, lo más conocidos son:

<h1></h1> permite definir encabezados (títulos), puede variar desde h1 hasta h6.

<p></p> define un párrafo de texto.

**** texto en negrita.

**** texto en negrita.

<i></i> texto en cursiva.

**** texto enfático en cursiva.

```
1 <!DOCTYPE html>
2 <html lang="es">
3
4 <head>
5     <meta charset="UTF-8" />
6     <title>Título de la Página</title>
7 </head>
8
9     <h1>Título en H1</h1>
10    <h2>Título en H2</h2>
11    <h3>Título en H3</h3>
12    <h4>Título en H4</h4>
13    <h5>Título en H5</h5>
14    <h6>Título en H6</h6>
15    <p>Párrafo</p>
16    <b>Texto en negrita</b>
17    <strong>Texto en negrita</strong>
18    <i>Texto en cursiva</i>
19    <em>Texto en cursiva</em>
20 <body>
21
22 </body>
23
24 </html>
```

Etiquetas de tablas

HTML permite la creación y personalización de tablas por medio de etiquetas muy simples y específicas:

<table></table> define la estructura contenedora de una tabla.

<tr></tr> define una fila en la tabla.

<td></td> define una celda de datos en una fila.

<th></th> define una celda de encabezado.

<thead></thead> agrupa los encabezados de la tabla.

<tbody></tbody> agrupa el cuerpo de la tabla (filas con datos).

```
1  <!DOCTYPE html>
2  <html lang="es">
3
4  <head>
5      <meta charset="UTF-8" />
6      <title>Título de la Página</title>
7  </head>
8
9  <body>
10     <table>
11         <thead>
12             <tr>
13                 <th>Nombre</th>
14                 <th>Materia</th>
15                 <th>Calificación</th>
16             </tr>
17         </thead>
18         <tbody>
19             <tr>
20                 <td>Ana</td>
21                 <td>Matemáticas</td>
22                 <td>95</td>
23             </tr>
24             <tr>
25                 <td>Carlos</td>
26                 <td>Historia</td>
27                 <td>89</td>
28             </tr>
29         </tbody>
30     </table>
31 </body>
32
33 </html>
```

Etiquetas de multimedia

HTML permite agregar elementos multimedia, como imágenes, audio y video, por medio de tres etiquetas:

**** permite agregar una imagen.

<audio>/<audio> permite agregar un control de audio.

<video>/<video> permite agregar un control de video.

Src es la fuente o ruta donde se encuentra el recurso multimedia.

```
1  <!DOCTYPE html>
2  <html lang="es">
3
4  <head>
5      <meta charset="UTF-8" />
6      <title>Título de la Página</title>
7  </head>
8
9  <body>
10     
11     <audio controls>
12         <source src="audio.mp3" type="audio/mpeg">
13     </audio>
14     <video controls>
15         <source src="video.mp4" type="video/mp4">
16     </video>
17 </body>
18
19 </html>
```

Etiquetas de formularios

Uno de los componentes más importantes de HTML es la posibilidad de crear formularios con todas las características y elementos necesarios.

<form></form> es el contendor de todo un formulario.

<input /> establece un campo de formulario de un tipo específico, algunos tipos son: texto, fechas, *e-mail*, *checkbox*, etc.

<select></select> establece un lista desplegable.

```
1  <!DOCTYPE html>
2  <html lang="es">
3
4  <head>
5      <meta charset="UTF-8" />
6      <title>Título de la Página</title>
7  </head>
8
9  <body>
10     <form action="registrar.php" method="post">
11         <label for="nombre">Nombre:</label>
12         <input type="text" id="nombre" name="nombre">
13         <input type="submit" value="Enviar">
14     </form>
15 </body>
16
17 </html>
```


Atributos comunes

A partir de las etiquetas más comunes vistas anteriormente, es importante entender que estas pueden tener unos atributos comunes y generales, algunos son:

id y **class** permiten identificar y agrupar elementos, sirven para integrar JavaScript y CSS.

src y **alt** permiten definir la fuente de un recurso y el texto alternativo al mismo.

Atributos específicos de algunas etiquetas: **type**, **value**, **method**, **action**.

```
1  <!DOCTYPE html>
2  <html lang="es">
3
4  <head>
5      <meta charset="UTF-8" />
6      <title>Título de la Página</title>
7  </head>
8
9  <body>
10     <form action="registrar.php" method="post">
11         <label for="nombre">Nombre:</label>
12         <input type="text" id="nombre" name="nombre">
13         <input type="submit" value="Enviar">
14     </form>
15 </body>
16
17 </html>
```



Practica lo aprendido

Pon en práctica lo visto sobre HTML resolviendo los siguientes ejercicios:

Ejercicio 1

Crea un formulario de contacto con los campos de: nombre, correo electrónico, mensaje y un botón para el envío.

Ejercicio 2

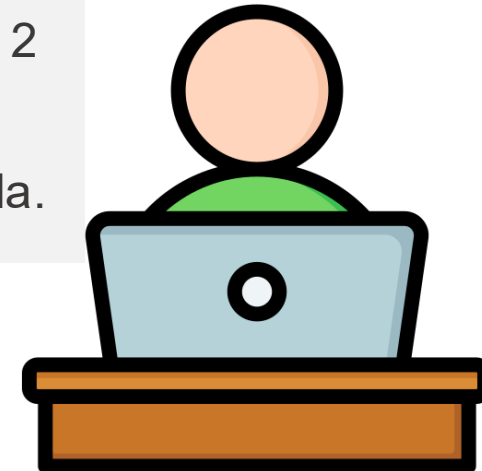
Crea una «galería» con al menos 3 fotos y 2 videos.

Ejercicio 4

Crea una hoja de vida con datos generales: estudios, experiencia, habilidades, etc.

Ejercicio 3

Crea una tabla de calificaciones con los campos de: nombre, documento, parcial 1, parcial 2, taller 1, taller 2 y promedio. Insertar al menos 5 filas en la tabla.



Tema 2:

Bootstrap

Bootstrap

Bootstrap es un *framework* de código abierto diseñado para facilitar la creación de sitios web *responsive* y estilos *atractivos* de manera *rápida* y eficiente. Ofrece una amplia colección de **componentes**, como botones, formularios, tablas y herramientas de diseño que simplifican el desarrollo de interfaces visuales **consistentes** y **modernas**. Fue creado originalmente por **Twitter** para asegurar uniformidad en sus herramientas internas (Get Bootstrap, 2025).

Estas son sus principales características, que veremos con detalle a continuación:

Diseño *responsive*

Compatibilidad

Integración

Extensible y personalizable

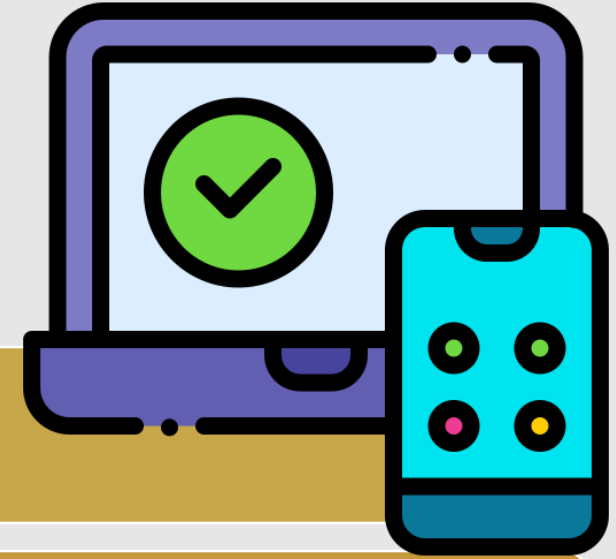


Imagen: Flaticon

Características



Imagen: Flaticon

Diseño *responsive*

Bootstrap fue diseñado desde el principio para que el contenido web se adapte a cualquier tamaño de pantalla, incluyendo dispositivos móviles, tabletas y monitores de escritorio. Esto se logra mediante un sistema de cuadrícula (*grid*) y una serie de clases que ajustan automáticamente el tamaño y el diseño de los elementos según el ancho de la pantalla.

Compatibilidad

Bootstrap está diseñado para funcionar de manera uniforme en todos los navegadores principales (Chrome, Firefox, Safari, Edge, entre otros). Al utilizar estándares web comunes, asegura que el sitio o aplicación web se renderice adecuadamente y se vea igual en diferentes entornos.

Características



Imagen: Flaticon

Integración

Bootstrap es fácil de implementar porque consiste en archivos CSS y JavaScript que se integran directamente en el HTML del proyecto.

Biblioteca de componentes.

Complementos de JavaScript (*plugins*).

Extensible y personalizable

Aunque Bootstrap ofrece un estilo base listo para usar, también es fácil de personalizar para adaptarse a las necesidades específicas del proyecto. Podemos modificar Bootstrap mediante:

- Clases personalizadas.
- Variables de entorno de CSS.
- Temas de Bootstrap.

¿Cómo integrarlo?

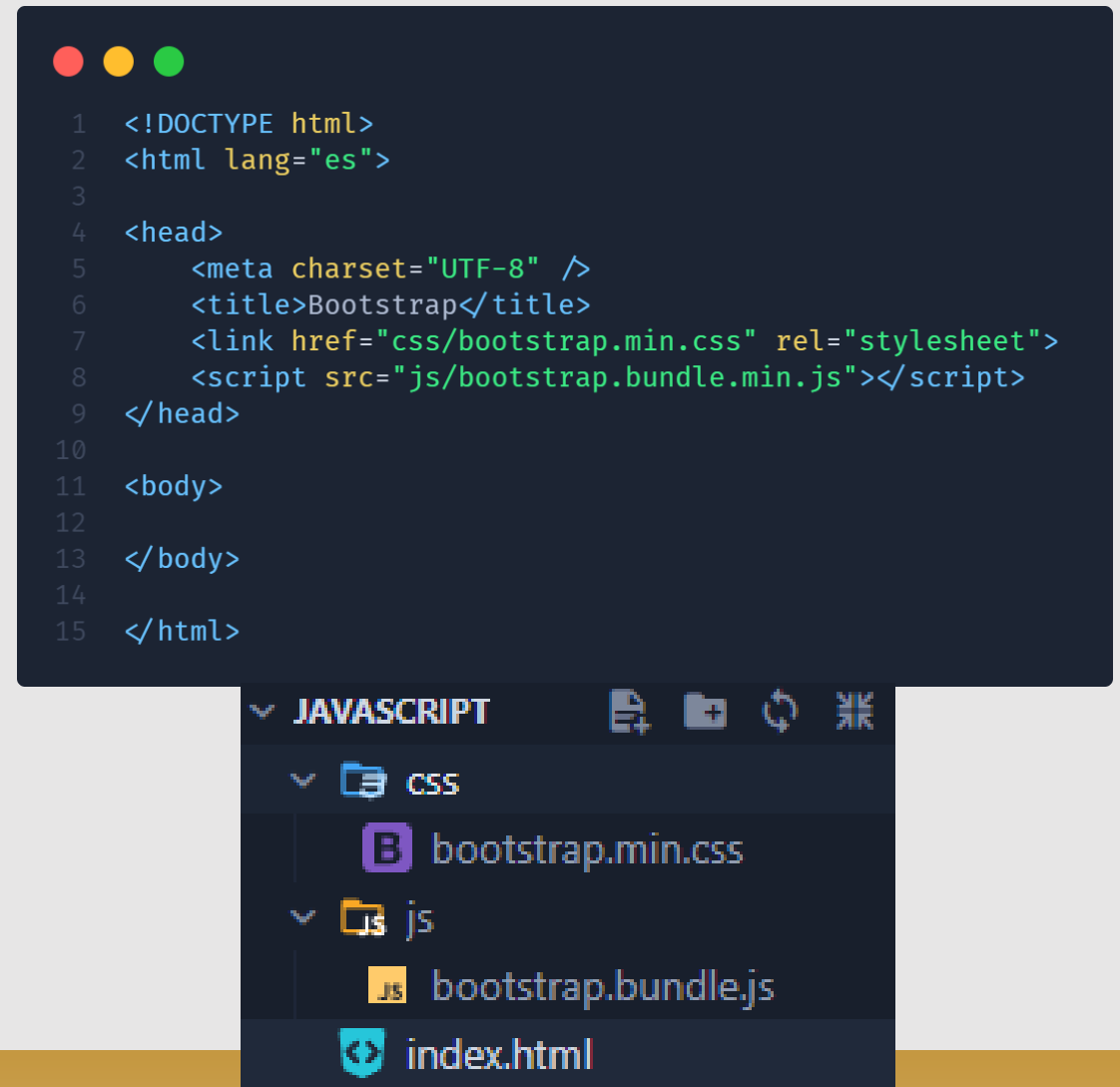
Hay dos formas comunes de integrar Bootstrap a los proyectos / páginas web.

A través de un **CDN**, integrando los componentes del **CSS** y JavaScript respectivamente.

Descargar directamente el **CSS** y **JavaScript** e incorporarlos al proyecto / página. Esta es la opción más recomendable porque, además, localmente se puede trabajar sin conexión a internet.

Para descargar:

<https://getbootstrap.com/docs/5.3/getting-started/download/>



Sistema grid

Bootstrap utiliza un sistema de cuadrícula de 12 columnas, lo que permite dividir el diseño en distintas secciones de manera flexible. La cuadrícula es **responsive**, por lo que se ajusta según el tamaño de la pantalla del dispositivo.

container: define un contenedor centralizado.

row: crea una fila de columnas.

col-[tamaño]: define el ancho de las columnas en distintos tamaños de pantalla.

```
1  <!DOCTYPE html>
2  <html lang="es">
3
4  <head>
5      <meta charset="UTF-8" />
6      <title>Bootstrap</title>
7      <link href="css/bootstrap.min.css" rel="stylesheet">
8      <script src="js/bootstrap.bundle.min.js"></script>
9  </head>
10
11 <body>
12     <div class="container">
13         <div class="row">
14             <div class="col-md-4">Columna 1</div>
15             <div class="col-md-4">Columna 2</div>
16             <div class="col-md-4">Columna 3</div>
17         </div>
18     </div>
19 </body>
20
21 </html>
```

Botones

Bootstrap ofrece una serie de clases personalizadas para establecer botones con diferentes estilos y funcionalidades.

```
1 <!DOCTYPE html>
2 <html lang="es">
3
4 <head>
5   <meta charset="UTF-8" />
6   <title>Bootstrap</title>
7   <link href="css/bootstrap.min.css" rel="stylesheet">
8   <script src="js/bootstrap.bundle.min.js"></script>
9 </head>
10
11 <body>
12   <button type="button" class="btn btn-primary">Botón Primario</button>
13   <button type="button" class="btn btn-secondary">Botón Secundario</button>
14   <button type="button" class="btn btn-success">Botón Éxito</button>
15   <button type="button" class="btn btn-danger">Botón Peligro</button>
16   <button type="button" class="btn btn-warning">Botón Advertencia</button>
17   <button type="button" class="btn btn-info">Botón Información</button>
18   <button type="button" class="btn btn-light">Botón Claro</button>
19 </body>
20
21 </html>
```

```
1 <!DOCTYPE html>
2 <html lang="es">
3
4 <head>
5   <meta charset="UTF-8" />
6   <title>Bootstrap</title>
7   <link href="css/bootstrap.min.css" rel="stylesheet">
8   <script src="js/bootstrap.bundle.min.js"></script>
9 </head>
10
11 <body>
12   <div class="alert alert-success" role="alert">
13     Este es un mensaje de éxito.
14   </div>
15   <div class="alert alert-danger" role="alert">
16     Este es un mensaje de error.
17   </div>
18   <div class="alert alert-warning" role="alert">
19     Este es un mensaje de advertencia.
20   </div>
21   <div class="alert alert-info" role="alert">
22     Este es un mensaje de información.
23   </div>
24   <div class="alert alert-primary" role="alert">
25     Este es un mensaje de color primario.
26   </div>
27   <div class="alert alert-secondary" role="alert">
28     Este es un mensaje de color secundario.</div>
29   <div class="alert alert-light" role="alert">
30     Este es un mensaje de color claro.
31   </div>
32 </body>
33
34 </html>
```

Alertas

Bootstrap ofrece una serie de clases personalizadas para establecer mensajes de alertas.

Barra de navegación

Bootstrap ofrece una serie de clases y etiquetas que fácilmente generan una interfaz de menú de navegación.

```
1 <!DOCTYPE html>
2 <html lang="es">
3
4 <head>
5   <meta charset="UTF-8" />
6   <title>Bootstrap</title>
7   <link href="css/bootstrap.min.css" rel="stylesheet">
8   <script src="js/bootstrap.bundle.min.js"></script>
9 </head>
10
11 <body>
12   <nav class="navbar navbar-expand-lg navbar-light bg-light">
13     <a class="navbar-brand" href="#">Mi Sitio</a>
14     <div class="collapse navbar-collapse">
15       <ul class="navbar-nav">
16         <li class="nav-item"><a class="nav-link" href="#">Inicio</a></li>
17         <li class="nav-item"><a class="nav-link" href="#">Contacto</a></li>
18       </ul>
19     </div>
20   </nav>
21 </body>
22
23 </html>
```



```
1 <!DOCTYPE html>
2 <html lang="es">
3
4 <head>
5   <meta charset="UTF-8" />
6   <title>Bootstrap</title>
7   <link href="css/bootstrap.min.css" rel="stylesheet">
8   <script src="js/bootstrap.bundle.min.js"></script>
9 </head>
10
11 <body>
12   <div class="card" style="width: 18rem;">
13     
14     <div class="card-body">
15       <h5 class="card-title">Título de la tarjeta</h5>
16       <p class="card-text">Descripción breve de la tarjeta.</p>
17       <a href="#" class="btn btn-primary">Más información</a>
18     </div>
19   </div>
20 </body>
21
22 </html>
```

Cards (tarjetas)

Bootstrap ofrece una serie de clases que permiten establecer una *card* (tarjeta) de contenido personalizable.

Clases útiles

Bootstrap incluye clases de utilidad que ayudan a dar estilo y espaciado sin escribir CSS personalizado:

Espaciado: **m** para margen y **p** para relleno.

mt-3: margen superior de tamaño 3.

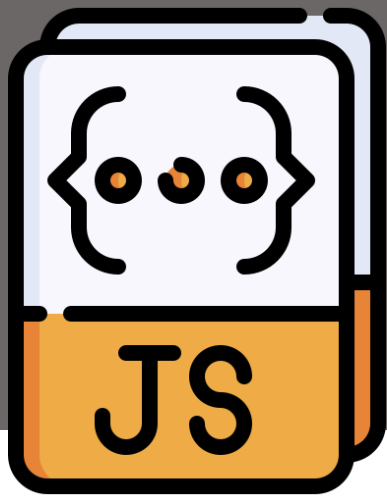
p-2: relleno de tamaño 2.

Colores de fondo: (bg-primary, bg-light, bg-dark).

Colores de texto: (text-primary, text-danger, text-success).

Alineación de texto: (text-center, text-start, text-end).





Practica lo aprendido

Para poner en práctica lo visto sobre Bootstrap, haz el siguiente ejercicio:

Ejercicio

Estiliza por medio de Bootstrap todos los ejercicios realizados con HTML. Ten libertad creativa.



Imagen: Freepik

Tema 3:

DOM (manipulación de HTML)

DOM

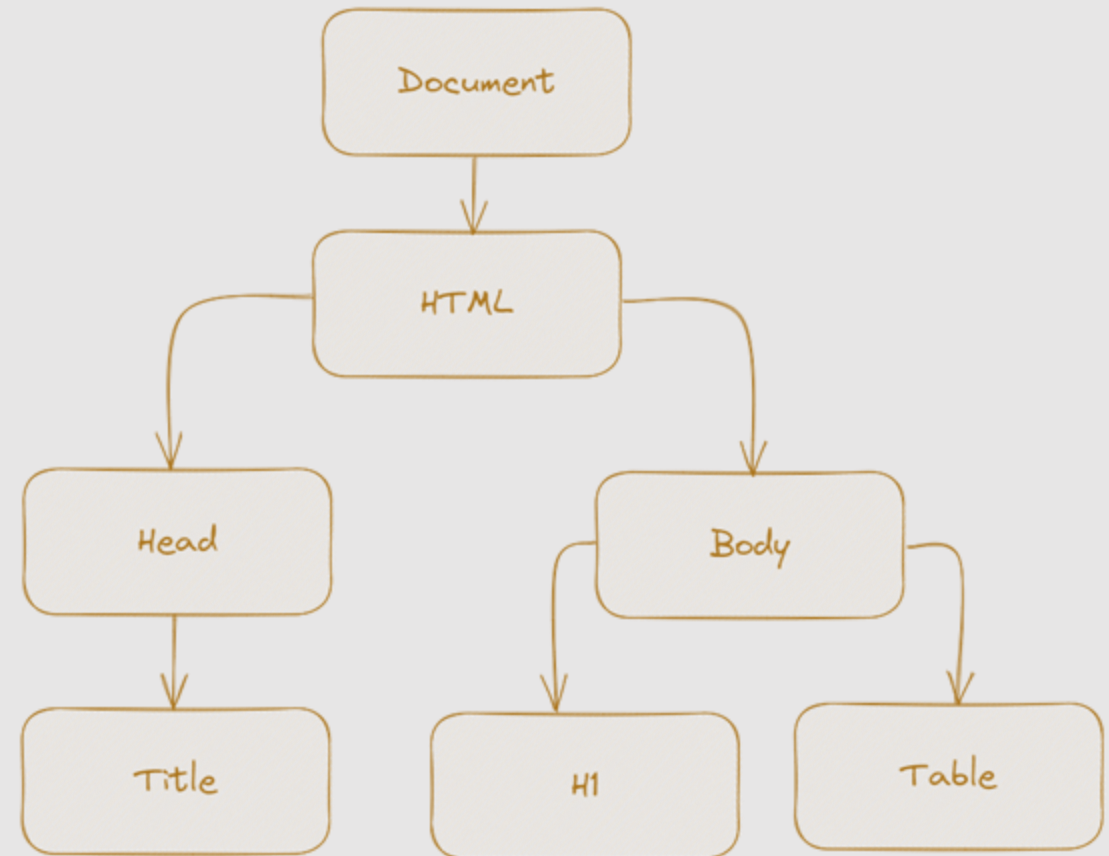
En el mundo del desarrollo web, normalmente comenzamos por aprender a escribir etiquetado o marcado **HTML** y, además, añadir estilos **CSS** para darle color, forma y algo de interacción. Sin embargo, a medida que avanzamos, es evidente que en cierta forma solo con esos dos componentes estamos bastante limitados y planos.

Si únicamente utilizamos **HTML/CSS**, solo podríamos crear páginas «estáticas» (sin demasiada personalización por parte del usuario), pero si añadimos JavaScript, podemos crear páginas «dinámicas».

Con páginas dinámicas hacemos alusión a que podemos dotar de la potencia y flexibilidad que da un lenguaje de programación a documentos y páginas para que sean mucho más ricos, que brinden una experiencia más completa y con el que se puedan automatizar tareas y acciones.

Las siglas **DOM** significan *document object model*, o lo que es lo mismo, la estructura del documento HTML. Una página HTML está formada por múltiples etiquetas HTML, anidadas una dentro de otra, formando un árbol de etiquetas relacionadas entre sí, que se denomina **árbol DOM** (o simplemente DOM).

El **DOM** se refiere a esta estructura que se puede modificar de forma dinámica desde JavaScript, añadiendo nuevas etiquetas, modificando o eliminando otras, cambiando sus atributos **HTML**, agregando clases, cambiando el contenido de texto, etc.



Document

En JavaScript, la forma de acceder al DOM es a través de un objeto llamado **document**, que representa el árbol DOM de la página o pestaña del navegador actual. En su interior pueden existir varios tipos de elementos, pero principalmente serán **Element** o **Node**.

Element: no es más que la representación genérica de una etiqueta: **HTMLElement**

Node: es una unidad más básica, la cual puede ser **Element** o un nodo de texto.

Todos los elementos **HTML**, dependiendo del elemento que sean, tendrán un tipo de dato específico:

Seleccionar etiquetas

Crear etiquetas

Insertar etiquetas

Gestión de CSS

Navegar

Seleccionar etiquetas

Si nos encontramos en nuestro código JavaScript y queremos hacer modificaciones en un elemento de la página HTML, lo primero que debemos hacer es buscar dicho elemento. Para ello, se suele intentar identificar el elemento a través de alguno de sus atributos más utilizados, generalmente el **id** o la **clase**.

querySelector permite seleccionar un único elemento que coincida con su selector.

querySelectorAll permite seleccionar varios elementos que coincidan con su selector.

permite acceder por medio de *ids* (identificadores)

Mientras que **.** permite acceder por medio de clases.

Métodos modernos

.querySelector()

.querySelectorAll()



```
1 const listaCursos = document.querySelector('#lista-cursos')
2
3 const cardsCursos = document.querySelectorAll('.card-curso')
4
5 const carrito = document.querySelector('#carrito')
6
7 const contenedorCarrito = document.querySelector('#lista-carrito tbody')
8
9 const links = document.querySelectorAll('a')
```

Crear elementos

Existe una serie de métodos para crear de forma eficiente diferentes elementos HTML o nodos, y que puede convertir en una tarea muy sencilla el crear estructuras dinámicas, mediante bucles o estructuras definidas:

```
1  const cardNew = document.createElement('div')
2
3  cardNew.classList.add('card')
4
5  cardNew.innerHTML = `
6    
7    <div class="card-body">
8      <h5 class="card-title">Card title</h5>
9      <p class="card-text">Lorem ipsum dolor sit am</p>
10     <a href="#" class="btn btn-primary">Go somewhere</a>
11   </div>
12 `
13
14  const cloneCard = cardNew.cloneNode(true)
15
```

Métodos modernos

`.createElement()`

`.cloneNode()`

createElement permite crear un elemento HTML a partir de su nombre (div, a, h1, h2, img, etc.).

cloneNode permite, a partir de un elemento creado, ser clonado en uno nuevo con todas sus características y componentes.

Atributos de elementos

Hasta ahora, hemos visto cómo crear elementos HTML con JavaScript, pero no hemos visto cómo **modificar** los atributos HTML de dichas etiquetas creadas.

En general, una vez tenemos un elemento sobre el que vamos a crear algunos atributos, lo más sencillo es asignarle valores como propiedades de objetos:

```
1  const div = document.createElement('div')
2
3  div.id = 'app'
4  div.className = 'data'
5  div.style = 'color: red'
6
7  // <div id="app" class="data" style="color: red"></div>
```

```
1  hasAttributes()
2  // Indica si el elemento tiene atributos HTML.
3
4  hasAttribute(attr)
5  // Indica si el elemento tiene el atributo HTML attr.
6
7  getAttributeNames()
8  // Devuelve un array con los atributos del elemento.
9
10 getAttribute(attr)
11 // Devuelve el valor del atributo attr del elemento o null si no existe.
12
13 removeAttribute(attr)
14 // Elimina el atributo attr del elemento.
15
16 setAttribute(attr, value)
17 // Añade o cambia el atributo attr al valor value.
18
19 getAttributeNode(attr)
20 // Devuelve el atributo attr como nodo.
21
22 removeAttributeNode(attr)
23 // Elimina el atributo attr del elemento y lo devuelve como nodo.
24
25 setAttributeNode(attr, value)
26 // Añade o cambia el atributo attr al valor value y lo devuelve como nodo.
```

Insertar elementos

Se trata de una vía rápida con la cual podemos añadir (o, más bien, **reemplazar**) el contenido de una etiqueta HTML.

```
1  const card = document.querySelector('.card')
2
3  card.textContent = 'Hello World'
4  // Cambia el texto del elemento
5
6  card.textContent // 'Hello World'
7  // Devuelve el texto del elemento
8
9  card.innerHTML = '<h1>Hello World</h1>'
10 // Cambia el HTML dentro del elemento
```

Uno de los métodos más comunes para añadir un elemento HTML creado con JavaScript es **appendChild()**.

Como su propio nombre indica, este método realiza un «append», es decir, inserta el elemento como un hijo al final de todos los elementos hijos que existan.

```
1  const card = document.querySelector('.card')
2
3  card.textContent = 'Politécnico de Colombia'
4  card.innerHTML = '<h2>Politécnico de Colombia</h2>'
5
6  const img = document.createElement('img')
7
8  img.src = 'img/politecnico.jpg'
9  img.alt = 'Logo Politécnico de Colombia'
10
11 card.appendChild(img)
```

Eliminar elementos

Probablemente, la forma más sencilla de eliminar nodos o elementos HTML es utilizando el método `.remove()` sobre el nodo o etiqueta para eliminar:

```
1 const card = document.querySelector('.card')
2
3 card.remove()
4
5 console.log(card) // null
```

El método `.remove()` se encarga de desconectarse del DOM a sí mismo.

Por último, con el método `.replaceChild()` se nos permite cambiar un nodo por otro.

```
1 const card = document.querySelector('.card')
2
3 const p = document.createElement('p')
4
5 p.textContent = 'I am a paragraph'
6
7 card.replaceChild(p, card.card)
8
9 console.log(card)
10
11 // <div class="card">
12 //   <p>I am a paragraph</p>
13 // </div>
```


Clases de elementos

JavaScript tiene a disposición una propiedad **.className** en todos los elementos HTML. Dicha propiedad contiene el valor del atributo **HTML class** como un **string**, y puede tanto leerse como reemplazarse:

La propiedad **.className** viene a ser la modalidad directa y rápida de utilizar el **getter** **.getAttribute("class")** y el **setter** **.setAttribute("class", v)**.

```
1  className
2  // Acceso directo al valor del atributo
3
4  classList
5  // Acceso directo a la lista de clases
6  // Contiene los siguientes métodos:
7
8  classList.length
9  // Número de clases
10 classList.item(index)
11 // Devuelve la clase en la posición index
12 classList.add(class1, class2, ... )
13 // Añade una o más clases
14 classList.remove(class1, class2, ... )
15 // Elimina una o más clases
16 classList.toggle(class)
17 // Añade la clase si no existe, la elimina si existe
18 classList.contains(class)
19 // Devuelve true si la clase existe, false si no
20 classList.replace(oldClass, newClass)
21 // Reemplaza la clase oldClass por newClass
```

Clases de elementos

Veamos un ejemplo utilizando estas propiedades y métodos y su equivalencia:

Para trabajar más cómodamente, existe un sistema muy interesante para trabajar con clases: el objeto **classList**. Se trata de un objeto especial que contiene una serie de métodos que permiten trabajar con las clases de forma más intuitiva y lógica.

```
1  const div = document.querySelector('.element')
2
3  div.className
4  // "element shine dark-theme"
5
6  div.getAttribute('class')
7  // "element shine dark-theme"
8
9  div.className = 'element brillo tema-oscuro'
10 // "element brillo tema-oscuro"
11
12 div.setAttribute('class', 'element brillo tema-oscuro')
```

Navegar elementos

En algunas ocasiones en las que conocemos y controlamos perfectamente la estructura del código HTML de la página, puede resultar más cómodo tener a disposición una serie de propiedades para navegar por la jerarquía de elementos HTML relacionados.

Las propiedades que veremos a continuación devuelven información de otros elementos relacionados con el elemento en cuestión.



```
1  children
2  // Devuelve un array con los hijos del nodo
3
4  parentElement
5  // Devuelve el nodo padre del nodo
6
7  firstElementChild
8  // Devuelve el primer nodo hijo del nodo
9
10 lastElementChild
11 // Devuelve el último nodo hijo del nodo
```

Tema 4:

Eventos

Eventos

Los eventos en JavaScript son acciones o sucesos que ocurren en una **página web**, como por ejemplo, hacer clic en un botón, mover el cursor del ratón, presionar una tecla del teclado, cargar una página o cambiar el tamaño de la ventana del navegador.

En JavaScript, los eventos son manejados mediante funciones conocidas como «manejadores de eventos» o «**event listeners**». Estas funciones se asocian con un elemento HTML específico (como un botón o un enlace) y se ejecutan automáticamente cuando ocurre el evento asociado con ese elemento.

click

mouseover

mouseout

keyup

keydown

DOMContentLoaded

addEventListener

Es un método en JavaScript que permite agregar un «**escucha**» o «**listener**» a un elemento del DOM para detectar cuándo ocurre un evento específico en ese elemento.

Este método se utiliza para asignar una función (conocida como «manejador de evento») a un evento específico que ocurre en un elemento HTML.

La sintaxis básica de **addEventListener** es la siguiente:



```
1 element.addEventListener(evento, callback)
2
3 // - element: es el elemento del DOM al que se le va a agregar el evento.
4 // - evento: es el nombre del evento que se le va a agregar al elemento.
5 // - callback: es la función que se va a ejecutar cuando se dispare el evento.
```



Proyecto

Proyecto

Para poner en práctica todos los conceptos vistos sobre JavaScript a lo largo del módulo, realizaremos el siguiente proyecto con todas las funcionalidades.

La idea es brindar esa propiedad dinámica aun sitio web diseñado en Bootstrap para hacerlo completamente funcional.

Los objetivos o funcionalidades deseadas son: contar con un formulario para el registro de productos en un arreglo de objetos para iterarlos y renderizarlos de manera mucho más eficiente en la vista del sitio web.

Nota: descargar la plantilla de “Tienda Virtual”.

- Crea un arreglo global que irá almacenando cada uno de los productos que se irán agregando.
- Añade la funcionalidad de agregar un producto al arreglo global con toda la información requerida.
- Crea la funcionalidad de renderizar los productos que se han registrado en el arreglo.
- Agrega un botón que permita eliminar un producto.
- Agregar un campo de buscador que permita filtrar los productos ya registrados.

Referencias bibliográficas

Get Bootstrap. (2025). *Cree sitios web rápidos y con capacidad de respuesta con Bootstrap*. Get Bootstrap. <https://getbootstrap.com/>

Get Bootstrap. (2025). *Descargar*. Get Bootstrap. <https://getbootstrap.com/docs/5.3/getting-started/download/>

Esta guía fue elaborada para ser utilizada con fines didácticos como material de consulta de los participantes en el diplomado virtual en PROGRAMACIÓN EN JAVASCRIPT del Politécnico de Colombia, y solo podrá ser reproducida con esos fines. Por lo tanto, se agradece a los usuarios referirla en los escritos donde se utilice la información que aquí se presenta.

GUÍA DIDÁCTICA 4

M3-DV114-GU04

MÓDULO 4: HTML, BOOTSTRAP Y DOM

© DERECHOS RESERVADOS - POLITÉCNICO DE
COLOMBIA, 2025
Medellín, Colombia

Proceso: Gestión Académica Virtual

Realización del texto: Diego Alejandro Palacio, docente

Revisión del texto: Comité de Revisión

Diseño: Comunicaciones

Editado por el Politécnico de Colombia

